

Phân tích dự đoán

- DFS/BFS đều là **giải thuật quay lui**.
- Đoán luật sinh được sử dụng, quay lui nếu không được.
- Cố gắng so khớp tiền tố để loại bỏ các câu không phù hợp.
- Còn có một lớp các giải thuật khác gọi là **giải thuật dự đoán (predictive algorithms)**.
- Dựa trên phần còn lại của đầu vào, dự đoán luật sinh (không cần quay lui).

Phân tích dự đoán

- Bộ phân tích dự đoán chạy rất **nhANH**.
- Nhiều giải thuật có độ phức tạp tuyến tính.
- **Sử dụng bảng** thay vì gọi đệ quy để cải thiện tốc độ.
- Tuy nhiên bộ phân tích dự đoán thường là **yếu**.
- Không phải tất cả các văn phạm đều có thể sử dụng được bộ phân tích dự đoán
- Dung hòa giữa **Khả năng biểu diễn (expressiveness)** và **tốc độ (speed)**.

Phân tích dự đoán

- Với ký hiệu bắt đầu, ta chọn luật sinh như thế nào ?
- Ý tưởng: sử dụng **các tokens nhìn thấy đầu tiên (lookahead tokens)**
- Khi quyết định chọn luật sinh nào, ta nhìn trước 1 (hoặc 1 số tokens) của đầu vào.

Cài đặt bộ phân tích dự đoán

- Phân tích dự đoán chỉ có thể sử dụng khi ta có thể **dự đoán chính xác luật sinh nào sẽ được sử dụng khi nhìn thấy trước một số tokens.**
- Tăng số lượng tokens nhìn thấy trước sẽ tăng số lượng văn phạm có thể áp dụng giải thuật này, nhưng ngược lại cũng làm tăng thêm độ phức tạp của giải thuật.
- Giảm số lượng tokens nhìn thấy trước sẽ làm giảm số lượng văn phạm có thể áp dụng, bù lại giải thuật sẽ đơn giản hơn.

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

E

$E \rightarrow T$

$E \rightarrow T + E$

$T \rightarrow \text{int}$

$T \rightarrow (E)$

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

E

$E \rightarrow T$

$E \rightarrow T + E$

$T \rightarrow \text{int}$

$T \rightarrow (E)$

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

E
T + E

E → **T**
E → **T + E**
T → **int**
T → **(E)**

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

E
T + E

E → **T**
E → **T + E**
T → **int**
T → **(E)**

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

E
T + E

E → **T**
E → **T + E**
T → **int**
T → **(E)**

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

E
$T + E$
$int + E$

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow int$
 $T \rightarrow (E)$

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

E
T + E
int + E

E → **T**
E → **T + E**
T → **int**
T → **(E)**

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

E
T + E
int + E

E → **T**
E → **T + E**
T → **int**
T → **(E)**

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

E
T + E
int + E

E → **T**
E → **T + E**
T → **int**
T → **(E)**

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

E
T + E
int + E
int + T

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

E
$T + E$
$int + E$
$int + T$

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow int$
 $T \rightarrow (E)$

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

E
T + E
int + E
int + T

E → **T**

E → **T** + **E**

T → **int**

T → (**E**)

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T + E
int + E
int + T
int + (E)

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T + E
int + E
int + T
int + (E)

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T + E
int + E
int + T
int + (E)

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T + E
int + E
int + T
int + (E)

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
$T + E$
$\text{int} + E$
$\text{int} + T$
$\text{int} + (E)$
$\text{int} + (T + E)$

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T + E
int + E
int + T
int + (E)
int + (T + E)

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T + E
int + E
int + T
int + (E)
int + (T + E)

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
$T + E$
$\text{int} + E$
$\text{int} + T$
$\text{int} + (E)$
$\text{int} + (T + E)$
$\text{int} + (\text{int} + E)$

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
$T + E$
$\text{int} + E$
$\text{int} + T$
$\text{int} + (E)$
$\text{int} + (T + E)$
$\text{int} + (\text{int} + E)$

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
$T + E$
$\text{int} + E$
$\text{int} + T$
$\text{int} + (E)$
$\text{int} + (T + E)$
$\text{int} + (\text{int} + E)$

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
$T + E$
$\text{int} + E$
$\text{int} + T$
$\text{int} + (E)$
$\text{int} + (T + E)$
$\text{int} + (\text{int} + E)$

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)
int + (int + T)

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)
int + (int + T)

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
$T + E$
$\text{int} + E$
$\text{int} + T$
$\text{int} + (E)$
$\text{int} + (T + E)$
$\text{int} + (\text{int} + E)$
$\text{int} + (\text{int} + T)$

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)
int + (int + T)
int + (int + int)

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
$T + E$
$\text{int} + E$
$\text{int} + T$
$\text{int} + (E)$
$\text{int} + (T + E)$
$\text{int} + (\text{int} + E)$
$\text{int} + (\text{int} + T)$
$\text{int} + (\text{int} + \text{int})$

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

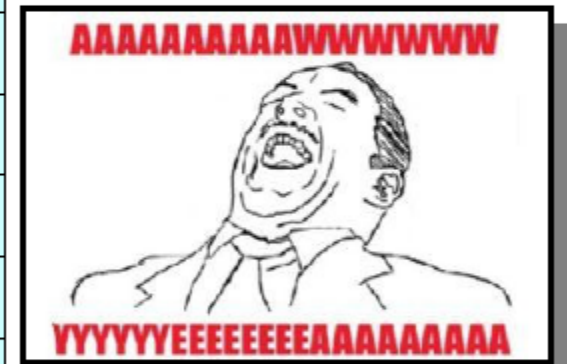
E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)
int + (int + T)
int + (int + int)

int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích dự đoán

$E \rightarrow T$
 $E \rightarrow T + E$
 $T \rightarrow \text{int}$
 $T \rightarrow (E)$

E
T + E
int + E
int + T
int + (E)
int + (T + E)
int + (int + E)
int + (int + T)
int + (int + int)



int	+	(	int	+	int	)
-----	---	---	-----	---	-----	---

Bộ phân tích LL(1)

- Phân tích từ trên xuống:
- • **L: Đọc từ trái sang phải**
- • **L: Dẫn xuất trái nhất.**
- • **(1): Nhìn trước 1 token**
- Xây dựng dẫn xuất trái nhất cho chuỗi các tokens.
- Khi triển khai một ký hiệu chưa kết thúc, ta dự đoán luật sinh được sử dụng bằng cách xem xét token nhìn thấy trước.

Bảng phân tích LL(1)

E $\rightarrow$ int

E $\rightarrow$ (E Op E)

Op $\rightarrow$ +

Op $\rightarrow$ *

	int	(	)	+	*
E	int	(E Op E)			
Op				+	*

(int + (int * int))

(1) **E** → int

(2) **E** → (**E Op E**)

(3) **Op** → +

(4) **Op** → *

E	(int + (int * int))
---	---------------------

(1) **E** → int

(2) **E** → (**E Op E**)

(3) **Op** → +

(4) **Op** → *

E	(int + (int * int))
---	---------------------

(1) **E** → int

(2) **E** → (**E Op E**)

(3) **Op** → +

(4) **Op** → *

	int	(	)	+	*
E	1	2			
Op				3	4

E\$

(int + (int * int))\$

(1) **E** → int

(2) **E** → (**E Op E**)

(3) **Op** → +

(4) **Op** → *

	int	(	)	+	*
E	1	2			
Op				3	4

- (1) **E** → **int**
- (2) **E** → (**E Op E**)
- (3) **Op** → **+**
- (4) **Op** → *****

	int	(	)	+	*
E	1	2			
Op				3	4

E\$	(int + (int * int))\$
-----	-----------------------

- \$ là ký hiệu đặc biệt thêm vào để đánh dấu kết thúc chuỗi nhập.
- Được sử dụng để biết khi nào thì kết thúc chuỗi nhập.
- \$ không thuộc văn phạm

E\$	(int + (int * int))\$
-----	-----------------------

- (1) **E** → int
- (2) **E** → (**E Op E**)
- (3) **Op** → +
- (4) **Op** → *

	int	(	)	+	*
E	1	2			
Op				3	4

- Ký hiệu đầu tiên trên đỉnh stack là ký hiệu chưa kết thúc **E**.
- Ký hiệu đầu tiên chuỗi nhập là (
- Ta tìm trong bảng phân tích (bên trái) luật sinh nào sẽ được sử dụng.
- Bước này gọi là **bước dự đoán**

E\$

(int + (int * int))\$

(1) **E** → int

(2) **E** → (**E Op E**)

(3) **Op** → +

(4) **Op** → *

	int	(	)	+	*
E	1	2			
Op				3	4

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

E\$	(int + (int * int))\$
(E Op E)\$	(int + (int * int))\$

	int	(	)	+	*
E	1	2			
Op				3	4

- (1) **E** → **int**
- (2) **E** → (**E Op E**)
- (3) **Op** → **+**
- (4) **Op** → *****

	int	(	)	+	*
E	1	2			
Op				3	4

E\$	(int + (int * int))\$
(E Op E)\$	(int + (int * int))\$

- Ký hiệu đầu tiên trên đỉnh stack bây giờ là ký hiệu kết thúc **)**.
- Ký hiệu đầu tiên của chuỗi nhập là **(**
- Hai ký hiệu này khớp với nhau, ta loại bỏ cả hai.
- Bước này gọi là **bước so khớp (match)**

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

E\$	(int + (int * int))\$
(E Op E)\$	(int + (int * int))\$
E Op E)\$	int + (int * int))\$

	int	(	)	+	*
E	1	2			
Op				3	4

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

E\$	(int + (int * int))\$
(E Op E)\$	(int + (int * int))\$
E Op E)\$	int + (int * int))\$

	int	(	)	+	*
E	1	2			
Op				3	4

- (1) **E** → **int**
- (2) **E** → (**E Op E**)
- (3) **Op** → **+**
- (4) **Op** → *****

	int	(	)	+	*
E	1	2			
Op				3	4

E\$	(int + (int * int))\$
(E Op E)\$	(int + (int * int))\$
E Op E)\$	int + (int * int))\$
int Op E)\$	int + (int * int))\$
Op E)\$	+ (int * int))\$
+ E)\$	+ (int * int))\$
E)\$	(int * int))\$
(E Op E))\$	(int * int))\$
E Op E))\$	int * int))\$
int Op E))\$	int * int))\$
Op E))\$	* int))\$
* E))\$	* int))\$
E))\$	int))\$
int))\$	int))\$
))\$	)\$
)\$	\$
\$	\$

Phát hiện lỗi cú pháp

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

int + int\$

	int	(	)	+	*
E	1	2			
Op				3	4

Phát hiện lỗi cú pháp

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

E\$	int + int\$
-----	-------------

	int	(	)	+	*
E	1	2			
Op				3	4

Phát hiện lỗi cú pháp

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

E\$	int + int\$
-----	-------------

	int	(	)	+	*
E	1	2			
Op				3	4

Phát hiện lỗi cú pháp

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

E\$	int + int\$
int \$	int + int\$

	int	(	)	+	*
E	1	2			
Op				3	4

Phát hiện lỗi cú pháp

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

E\$	int + int\$
int \$	int + int\$
\$	+ int\$

	int	(	)	+	*
E	1	2			
Op				3	4

Phát hiện lỗi cú pháp

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

E\$	int + int\$
int \$	int + int\$
\$	+ int\$

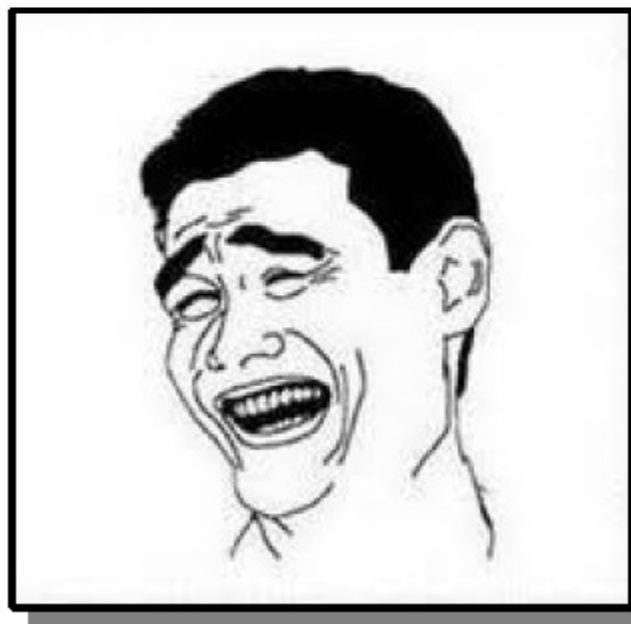
	int	(	)	+	*
E	1	2			
Op				3	4

Phát hiện lỗi cú pháp

- (1) **E** $\rightarrow$ **int**
- (2) **E** $\rightarrow$ (**E Op E**)
- (3) **Op** $\rightarrow$ **+**
- (4) **Op** $\rightarrow$ *****

E\$	int + int\$
int \$	int + int\$
\$	+ int\$

	int	(	)	+	*
E	1	2			
Op				3	4



Phát hiện lỗi cú pháp

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

(int (int))\$

	int	(	)	+	*
E	1	2			
Op				3	4

Phát hiện lỗi cú pháp

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

E\$	(int (int))\$
-----	---------------

	int	(	)	+	*
E	1	2			
Op				3	4

Phát hiện lỗi cú pháp

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

E\$	(int (int))\$
-----	---------------

	int	(	)	+	*
E	1	2			
Op				3	4

Phát hiện lỗi cú pháp

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

E\$	(int (int))\$
(E Op E)\$	(int (int))\$

	int	(	)	+	*
E	1	2			
Op				3	4

Phát hiện lỗi cú pháp

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

E\$	(int (int))\$
(E Op E)\$	(int (int))\$
E Op E)\$	int (int))\$

	int	(	)	+	*
E	1	2			
Op				3	4

Phát hiện lỗi cú pháp

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

E\$	(int (int))\$
(E Op E)\$	(int (int))\$
E Op E)\$	int (int))\$

	int	(	)	+	*
E	1	2			
Op				3	4

Phát hiện lỗi cú pháp

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

E\$	(int (int))\$
(E Op E)\$	(int (int))\$
E Op E)\$	int (int))\$
int Op E)\$	int (int))\$

	int	(	)	+	*
E	1	2			
Op				3	4

Phát hiện lỗi cú pháp

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

E\$	(int (int))\$
(E Op E)\$	(int (int))\$
E Op E)\$	int (int))\$
int Op E)\$	int (int))\$
Op E)\$	(int))\$

	int	(	)	+	*
E	1	2			
Op				3	4

Phát hiện lỗi cú pháp

(1) **E** → **int**

(2) **E** → (**E Op E**)

(3) **Op** → **+**

(4) **Op** → *****

	int	(	)	+	*
E	1	2			
Op				3	4

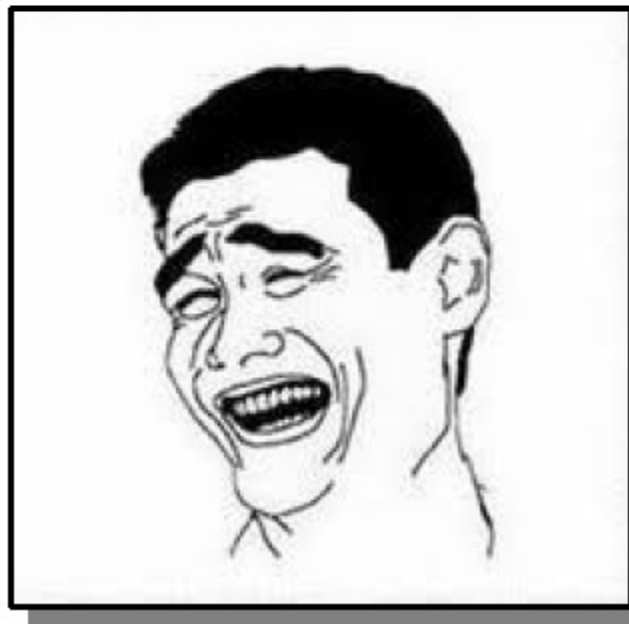
E\$	(int (int))\$
(E Op E)\$	(int (int))\$
E Op E)\$	int (int))\$
int Op E)\$	int (int))\$
Op E)\$	(int))\$

Phát hiện lỗi cú pháp

- (1) **E** $\rightarrow$ **int**
- (2) **E** $\rightarrow$ (**E Op E**)
- (3) **Op** $\rightarrow$ **+**
- (4) **Op** $\rightarrow$ *****

	int	(	)	+	*
E	1	2			
Op				3	4

E\$	(int (int))\$
(E Op E)\$	(int (int))\$
E Op E)\$	int (int))\$
int Op E)\$	int (int))\$
Op E)\$	(int))\$



Giải thuật phân tích LL(1)

- Giả sử văn phạm có ký hiệu bắt đầu là S và bảng phân tích LL(1) là T . Ta muốn phân tích chuỗi nhập ω
- Khởi tạo stack chứa **$S\$$** .
- Lặp lại cho đến khi stack rỗng:
 - Gọi ký tự đầu tiên của ω là t .
 - Nếu đỉnh stack là ký hiệu kết thúc r :
 - Nếu r và t không khớp với nhau $\rightarrow$ Lỗi.
 - Ngược lại, loại bỏ t ra khỏi ω , bỏ r ra khỏi stack.
 - Ngược lại, gọi ký hiệu chưa kết thúc trên đỉnh stack là A :
 - Nếu $T[A, t]$ rỗng (undefined) $\rightarrow$ Lỗi.
 - Ngược lại, loại bỏ A ra khỏi stack, thêm $T[A, t]$ vào stack.

Ví dụ

STMT → if **EXPR** then **STMT**
| while **EXPR** do **STMT**
| **EXPR** ;

EXPR → **TERM** -> id
| zero? **TERM**
| not **EXPR**
| ++ id
| -- id

TERM → id
| constant

Ví dụ

STMT → if **EXPR** then **STMT**
| while **EXPR** do **STMT**
| **EXPR** ;

EXPR → **TERM** -> id id -> id;
| zero? **TERM** while not zero? id
| not **EXPR** do --id;
| ++ id if not zero? id then
| -- id if not zero? id then
 constant -> id;
TERM → id
| constant

Xây dựng bảng phân tích LL(1)

STMT $\rightarrow$ **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR ;** (3)

EXPR $\rightarrow$ **TERM** $\rightarrow$ **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM $\rightarrow$ **id** (9)
 | **constant** (10)

	if	then	while	do	zero?	not	++	--	$\rightarrow$	id	const	;
STMT												
EXPR												
TERM												

Xây dựng bảng phân tích LL(1)

STMT $\rightarrow$ **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR $\rightarrow$ **TERM** $\rightarrow$ **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM $\rightarrow$ **id** (9)
 | **constant** (10)

	if	then	while	do	zero?	not	++	--	$\rightarrow$	id	const	;
STMT												
EXPR												
TERM												

Xây dựng bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** -> **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT												
EXPR												
TERM										9	10	

Xây dựng bảng phân tích LL(1)

STMT $\rightarrow$ **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR $\rightarrow$ **TERM** $\rightarrow$ **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM $\rightarrow$ **id** (9)
 | **constant** (10)

	if	then	while	do	zero?	not	++	--	$\rightarrow$	id	const	;
STMT												
EXPR												
TERM										9	10	

Xây dựng bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT												
EXPR					5	6	7	8				
TERM										9	10	

Xây dựng bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT												
EXPR					5	6	7	8				
TERM										9	10	

Xây dựng bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT												
EXPR					5	6	7	8		4	4	
TERM										9	10	

Xây dựng bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT												
EXPR					5	6	7	8		4	4	
TERM										9	10	

Xây dựng bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** -> **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2									
EXPR					5	6	7	8		4	4	
TERM										9	10	

Xây dựng bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** -> **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2									
EXPR					5	6	7	8		4	4	
TERM										9	10	

Xây dựng bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** -> **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3				
EXPR					5	6	7	8		4	4	
TERM										9	10	

Xây dựng bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3				
EXPR					5	6	7	8		4	4	
TERM										9	10	

Xây dựng bảng phân tích LL(1)

STMT → **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR** ; (3)

EXPR → **TERM** → **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM → **id** (9)
 | **constant** (10)

	if	then	while	do	zero?	not	++	--	→	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR					5	6	7	8		4	4	
TERM										9	10	

Xây dựng bảng phân tích LL(1)

STMT $\rightarrow$ **if** **EXPR** **then** **STMT** (1)
 | **while** **EXPR** **do** **STMT** (2)
 | **EXPR ;** (3)

EXPR $\rightarrow$ **TERM** $\rightarrow$ **id** (4)
 | **zero?** **TERM** (5)
 | **not** **EXPR** (6)
 | **++ id** (7)
 | **-- id** (8)

TERM $\rightarrow$ **id** (9)
 | **constant** (10)

	if	then	while	do	zero?	not	++	--	$\rightarrow$	id	const	;
STMT	1		2		3	3	3	3		3	3	
EXPR					5	6	7	8		4	4	
TERM										9	10	

Giải thuật xây dựng bảng LL(1)

- Trực quan: ký tự kế tiếp phải duy nhất để có thể xác định được luật sinh, vì vậy ta sẽ chọn luật sinh mà nó sinh ra chuỗi bắt đầu bằng ký tự đang xét.
- Phần tử $T[A, t]$ là luật sinh có dạng $A \rightarrow \omega$ *khí và chỉ khí* ω dẫn xuất ra chuỗi bắt đầu bằng ký tự t .
- Chặt chẽ hơn:
$$T[A, t] = B\omega \text{ *khí và chỉ khí* } A \rightarrow \omega \text{ và } \omega \Rightarrow^* t\omega'$$